Airport Runway Scheduling System

Data Structures & Java Project — Comprehensive Viva Voce Defense Guide

1. Project Core Concepts (Elevator Pitch)

SYSTEM PURPOSE

Automates a single-runway airport traffic controller where arriving and departing aircraft share a unified queue. Emergency flights instantly preempt routine traffic.

DATA STRUCTURE CHOICE

Custom **Singly Linked List Priority Queue** implemented from scratch without `java.util`. Enqueue and Dequeue operate in constant **O(1)** time.

Operation	Queue Action	Time Complexity	Aviation Rule
Routine Flight	Inserted at Tail (`rear`)	**O(1)**	Standard FIFO (First-In, First-Out) order
Emergency Flight	Inserted at Head (`front`)	**O(1)**	Safety Priority (Bypasses routine line)
Runway Dispatch	Removed from Head (`front`)	**O(1)**	Clears highest-priority waiting flight

2. Individual Member Modules (Part 1)

Justin Joe Mathew `Aircraft.java`

Aircraft Management & Java OOP

> **Core Responsibility:** Encapsulates flight attributes (`flightNo`, `airline`, `type`, `isEmergency`), constructor initialization, and formatted text output via `toString()`.

Q1: Which Object-Oriented Programming (OOP) concepts are demonstrated in Aircraft.java?

- **Encapsulation** (packaging related flight attributes into a single cohesive class) and **Polymorphism** (overriding the `toString()` method from the parent `Object` class).

Q2: Why did you override toString()?

- It cleanly formats the flight as `[EMERGENCY] AI-101 (Air India)` or `AI-101 (Air India)` so the UI and logs can render flight objects directly without manual string concatenation.

30-Second Viva Speech: "I developed the Aircraft class using OOP encapsulation to store flight details and overridden toString() to provide clean, formatted representations across our Swing GUI and persistence logs."

Elna Elizabeth Abraham `Node.java` `FlightQueue.java`

Data Structures & Core Algorithms

> **Core Responsibility:** Built a custom Singly Linked-List Priority Queue with `front` and `rear` pointers from scratch. Zero reliance on `java.util` collections.

Q1: Why did you implement a Linked List instead of an Array for the Queue?

- Dynamic memory allocation (no static size boundary) and **O(1)** insertion/deletion without needing to shift elements like an array.

Q2: How does your Priority Queue logic work without loops?

- If an incoming flight is routine, `rear.next = newNode; rear = newNode;` (tail insertion). If it's an emergency, `newNode.next = front; front = newNode;` (head insertion). Both execute in **O(1)** time.

30-Second Viva Speech: "I engineered our custom Priority Queue using a Singly Linked List with front and rear pointers. Routine flights join at the tail in FIFO order, while emergency flights jump directly to the head. All queue operations execute in O(1) time."

Airport Runway Scheduling System

Individual Modules & Team Presentation Script

2. Individual Member Modules (Part 2)

K Sruthika `RunwayUI.java` — UI & System Integration

> **Core Responsibility:** Designed the classic Swing GUI with `JFrame`, `setBounds` (null layout), and `implements ActionListener` to connect the visual interface to the underlying data structure.

Q1: How does the UI connect with the Queue data structure?

- When the user clicks 'Add Flight', it triggers `runwayQueue.enqueue(flight)`. When 'Clear Runway' is clicked, it calls `runwayQueue.dequeue()` and refreshes the display text areas.

Q2: Why use a single unified queue on the interface?

- Because in a single-runway airport, all arriving and departing aircraft compete for the same physical runway. Having one list makes priority preemption immediately visible to air traffic controllers.

30-Second Viva Speech: "I built the graphical interface using pure Java Swing with null layout and setBounds, matching our lab coursework. My module captures user input, updates the queue visually, and dispatches flights via button events."

Devika Suresh `FileHandler.java` `TestRunway.java` — File Handling & Testing

> **Core Responsibility:** Manages persistent logging to `runway_log.txt` using `FileWriter` in append mode and created automated headless test verification scripts.

Q1: Why did you use append mode (new FileWriter(FILE_NAME, true))?

- Passing `true` prevents overwriting past flight history, ensuring an audit log of all runway clearances is permanently preserved on disk.

Q2: What edge cases did you verify with TestRunway.java?

- 1. Queue underflow (attempting to clear an empty runway shows an alert without crashing). 2. Verifying that emergency flights inserted after routine flights dequeue first.

30-Second Viva Speech: "I implemented persistent audit logging using FileWriter in append mode and built TestRunway.java to programmatically verify our priority queue logic and underflow protection without needing the GUI."

3. Team Presentation Flow (How to Present to the Examiner)

1. **Justin introduces the project:** "Good morning sir/ma'am. We have developed an Airport Runway Scheduling System in Java modeling a priority-based single runway controller. I designed the flight OOP structure in `Aircraft.java`."
2. **Elna explains the Data Structure:** "Instead of using Java's built-in collections, I implemented a Singly Linked-List Priority Queue from scratch. Emergency flights insert at Head (O(1)), normal flights insert at Tail (O(1)), and runway dispatch removes from Head (O(1))."
3. **Sruthika demonstrates the Live GUI:** "I built our Swing GUI. (Run `./run.sh`). Watch as I add flight AI-101 (Landing), then 6E-202 (Takeoff), and now EM-999 with Emergency checked. Notice how EM-999 jumps directly to Position #1 in the queue!"
4. **Devika concludes with Persistence & Tests:** "Clicking 'Clear Runway' clears EM-999 first. I built `FileHandler` which writes every event to `runway_log.txt`, and `TestRunway.java` which automated our verification."